Indian Institute of Technology Jodhpur

Department of Computer Science and Engineering

M.Tech. in Data and Computational Sciences

Machine Learning for Big Data

Project Report

Scalable Near-Duplicate Detection on Wikipedia Using Apache Spark and Locality-Sensitive Hashing

Submitted by:

Basu Singh (M24AID008)
Nitin Jain (M24AID022)
Shashi Saurav (M24AID048)

GitHub Link:
https://github.com/Yoursuperherobasu/BigData_Project_Machine_Learning-_for_Big_Data.git

Table of Contents

1. Introduction

2. Literature Review

3. Problem Statement and Objectives

4. System Architecture and Design

5. Dataset Description

6. Implementation Details

   6.1 Data Ingestion

   6.2 Text Pre-processing and Feature Engineering

   6.3 MinHash and Locality-Sensitive Hashing

   6.4 Candidate Pair Generation

   6.5 Evaluation Framework

7. Experimental Setup

8. Results and Analysis

   8.1 Scalability Analysis

   8.2 Accuracy Evaluation

9. Testing Strategy

10. How to Reproduce

11. Challenges and Lessons Learned

12. Conclusion and Future Work

13. References

1. Introduction

Wikipedia has over six million articles in English alone, and a surprising number of them overlap with each other. We noticed this when we started exploring the dataset for this project. Some articles are almost word-for-word copies of others, usually because someone forked an existing page to create a new one, or because two articles that covered similar topics got partially merged at some point. Editors also tend to copy entire paragraphs across related pages. All this duplication is a real headache for anyone trying to do NLP tasks on Wikipedia data, because your search index or classifier ends up treating the same content as if it were independent.

The obvious approach to finding duplicates is to compare every article against every other article, but that does not work here. With n articles the number of pairs is n*(n-1)/2, which for six million articles comes out to something like 18 trillion comparisons. Even on a fast machine that would take an unreasonable amount of time. And the data itself is too big to hold in RAM on one computer, so you cannot just load everything and loop through it.

We decided to use Apache Spark with the MinHash LSH algorithm from Spark MLlib to get around both of these problems. The idea behind MinHash is that you can estimate how similar two documents are by hashing their word sets and comparing the hash values, rather than actually computing the intersection and union. Then LSH takes it one step further by sorting documents into buckets based on their hash signatures, so you only need to compare documents that ended up in the same bucket. The upshot is that the cost goes from quadratic down to something much more manageable, and Spark takes care of distributing the work across cores.

For our experiments we used a Wikipedia dump from March 2026, specifically one multistream shard that covers about 100,000 articles after cleanup. The pipeline reads the raw XML, strips out the junk (redirects, stub pages, non-article namespaces), converts each article into a binary feature vector, runs MinHash LSH, and writes out the detected near-duplicate pairs with their Jaccard distances. We then ran two kinds of evaluations: one to see how runtime changes as we feed in more data, and one to see how the LSH results compare to doing a full brute-force comparison on a small sample.

2. Literature Review

Duplicate detection in document collections has been studied since the late 1990s. Broder (1997) is probably the paper that started it all. His idea was to represent each document as the set of its word shingles (basically overlapping subsequences) and then use Jaccard similarity to compare them. Jaccard similarity is just the number of common shingles divided by the total number of distinct shingles across both documents. It sounds simple, and it is, but it turned out to be surprisingly effective for catching near-duplicates.

The problem with plain Jaccard is that computing it for every pair of documents is expensive. MinHash, which Broder also came up with (formalised later with Charikar, Frieze, and Mitzenmacher in 1998), solves this. You take k different random hash functions, apply each one to every element in the shingle set, and keep only the minimum hash value per function. That gives you a short signature of k numbers. Here is the clever part: the probability that two documents share the same minimum hash for any one function is exactly their Jaccard similarity. So the fraction of matching positions in two signatures is a decent estimate, and it gets better as you increase k.

LSH was formalised by Indyk and Motwani (1998) as a general trick for approximate nearest neighbour search. When you pair it with MinHash, you chop the k-length signature into b bands of r rows each. A pair of documents becomes a candidate only if they match on every single row within at least one band. The math works out to a probability of $1-(1-s^r)^b$, where s is the true similarity. If you plot this for different b and r values you get an S-shaped curve, and by adjusting b and r you can control how aggressive the filtering is.

Others have applied similar ideas at even larger scale. Henzinger (2006) tested fingerprinting techniques on billions of pages at Google. Manku, Jain, and Sarma (2007) proposed SimHash for web crawl deduplication. On the engineering side, Spark has made it much more practical to run these algorithms because you do not need a custom distributed system anymore. Spark MLlib comes with MinHashLSH ready to use, and it works on DataFrames, so the integration is fairly smooth.

The textbook by Leskovec, Rajaraman, and Ullman (Mining of Massive Datasets, 3rd edition, Chapter 3) covers the theory in detail and we referenced it heavily while designing our

pipeline. What our project adds is an empirical evaluation on actual Wikipedia data, with concrete numbers on both scaling and accuracy.

3. Problem Statement and Objectives

Our input is a collection of Wikipedia articles stored as MediaWiki XML. The goal is to find all pairs of articles that are near-duplicates of each other. We define "near-duplicate" as having a Jaccard distance below a threshold that we can configure (we used 0.3 for most experiments). The system needs to work with at least a hundred thousand articles and should finish in a reasonable time frame on a regular multi-core machine.

The specific objectives we set for ourselves were:

- Build an end-to-end pipeline that goes from raw Wikipedia XML to a list of near-duplicate pairs, running entirely on Spark.
- Use MinHash and LSH so that we do not have to do O(n^2) pairwise comparisons, which would be impossibly slow at this scale.
- Measure how the pipeline's runtime grows as we give it more data, by running it at different sample sizes.
- Verify the accuracy of the LSH approach by comparing it against a brute-force all-pairs Jaccard computation on a small subset, and reporting precision, recall, and F1.
- Keep the code modular and documented enough that someone else could clone the repo and reproduce our results, either locally or on an HDFS cluster.

4. System Architecture and Design

We split the system into five stages, each in its own Python file inside the wiki_near_dup package. The stages are: ingestion, feature engineering, LSH fitting, candidate pair generation, and evaluation. Having them separate made debugging a lot easier because we could test each part on its own without running the whole pipeline.

The high-level data flow looks like this:

Wikipedia XML Dump --> Spark XML Parser --> Article Normalisation --> Tokenisation + HashingTF --> Binary Feature Vectors --> MinHash LSH (Fit + Transform) --> Self-Join on Candidate Buckets --> Jaccard Distance Filtering --> Output Parquet (Near-Duplicate Pairs)

Everything runs on Spark using DataFrames. We chose DataFrames over RDDs because the Catalyst optimiser handles a lot of the query planning for you, and the code is easier to read. For parsing the Wikipedia XML we use spark-xml, which reads the XML directly into a DataFrame without any external preprocessing step. One thing we had to be careful about was memory: we cache DataFrames after expensive operations and call unpersist as soon as we no longer need them. Without this the pipeline would crash on larger samples because Spark would try to recompute things it had already thrown away.

The table below shows what each module is responsible for:

Module	Responsibility
ingest.py	XML parsing via spark-xml, article filtering (namespace, redirects, minimum text length), and sampling
features.py	Tokenisation (RegexTokenizer) and binary feature vector construction (HashingTF in binary mode)
lsh_pipeline.py	MinHashLSH model fitting and approximate similarity self-join for candidate pair generation
evaluate.py	Scalability study (runtime vs. data volume) and accuracy evaluation (precision, recall, F1 vs. brute-force)
jaccard_utils.py	Pure-Python Jaccard similarity, distance, brute-force pairwise comparison, and precision/recall/F1 helpers
config.py	Default hyperparameters and Maven coordinates for spark-xml
run_pipeline.py	CLI entry point with argparse; orchestrates all

	pipeline stages
spark_submit.sh	Shell wrapper that sets PYTHONPATH and passes the spark-xml Maven package to spark-submit

5. Dataset Description

We downloaded one multistream shard from the March 2026 English Wikipedia dump. The specific file is enwiki-20260301-pages-articles-multistream2.xml-p41243p151573.bz2, covering page IDs 41,243 through 151,573. It is in MediaWiki XML format compressed with bzip2.

The compressed file weighs about 380 MB. After parsing and filtering out redirect pages, non-article namespaces, and articles with fewer than 50 characters of text, we were left with a little over 100,000 articles. Each article has three fields: page_id, title, and the full text content.

We could have used the full Wikipedia dump but that would have been impractical for running experiments on a laptop. One shard was big enough to give us meaningful results while still being feasible for local execution. The pipeline code does not care how many shards you give it though, so moving to the full dump on HDFS would just mean pointing it at a different path.

Property	**Value**
Source	Wikimedia Foundation dumps (dumps.wikimedia.org)
Dump date	20260301 (March 2026)
File	enwiki-20260301-pages-articles-multistream2.xml-p41243p151573.bz2
Compressed size	~380 MB
Format	MediaWiki XML inside bzip2

Page ID range	41,243 to 151,573
Articles after filtering	~100,000+
Article schema	page_id (long), title (string), text (string)

6. Implementation Details

6.1 Data Ingestion

ingest.py handles reading the Wikipedia XML. We use the spark-xml library (com.databricks:spark-xml_2.12:0.18.0) with the rowTag set to "page" and parse mode set to PERMISSIVE. This reads every <page> element into a row of a Spark DataFrame. We went with this approach because it lets us do everything inside one Spark job instead of having a separate script to convert the XML first.

The raw DataFrame needs a fair bit of cleaning. We filter to namespace 0, which is the main article namespace; everything else (Talk pages, User pages, Wikipedia internal pages) gets dropped. Redirects are easy to spot because their text content starts with "#REDIRECT", so we filter those out too. Finally we remove anything with less than 50 characters of text, which gets rid of stubs and empty pages. What remains is a clean DataFrame with page_id, title, and text columns.

There is also a sampling function that takes a fraction between 0 and 1 and a seed. We use seed 42 everywhere so the same fraction always gives the same sample. This was important for the scaling experiments because we wanted the results to be reproducible.

6.2 Text Pre-processing and Feature Engineering

features.py turns the raw article text into sparse binary vectors. We set it up as a two-stage Spark ML Pipeline because that is the cleanest way to chain transformations in PySpark.

Stage one is a RegexTokenizer with the pattern \W+ (any non-word character). It lowercases everything and throws away tokens shorter than 2 characters. We did not bother with stemming or lemmatisation because Jaccard on word sets already does a decent job for our

purpose, and adding NLP preprocessing would have slowed things down without a clear accuracy benefit.

Stage two is HashingTF in binary mode. Instead of counting word frequencies it just sets each hash bucket to 1 if the word is present and 0 if it is not. The hash space has 2^18 = 262,144 dimensions. We tried 2^16 first but got too many hash collisions, and 2^20 used more memory without noticeably improving results, so 2^18 seemed like the right spot. Binary mode is necessary because Jaccard similarity is defined on sets (is the token there or not), not on bags (how often it appears).

The output is a DataFrame where each row has an additional "features" column containing a SparseVector. These vectors go directly into the MinHash LSH model.

6.3 MinHash and Locality-Sensitive Hashing

lsh_pipeline.py is where the actual deduplication happens. It wraps Spark MLlib's MinHashLSH class. What MinHash does is take each article's set of non-zero feature indices, apply several random hash functions to them, and keep the minimum value for each function. The result is a compact signature per document.

The mathematical reason this works is that for two sets A and B, the probability of their min-hash values agreeing under a single random hash function equals their Jaccard similarity: P(min h(A) = min h(B)) = |A intersection B| / |A union B|. With k independent hash functions you get a k-dimensional signature, and the agreement rate across those k positions is an unbiased estimator of the true similarity. The estimation error shrinks as 1/sqrt(k).

LSH adds a banding trick on top. The signature gets divided into b bands, and two documents become candidates if they agree on every position within at least one band. Documents with high similarity will almost certainly end up as candidates, while dissimilar ones almost certainly will not. Spark MLlib exposes this through the numHashTables parameter, which controls the number of bands. We set it to 5 after some initial experimentation.

6.4 Candidate Pair Generation

For the actual pair generation we call approxSimilarityJoin from Spark MLlib. You pass it the featurised DataFrame and a distance threshold (we used 0.3, so we are looking for pairs with at least 70% similarity). It does a self-join internally and returns all pairs below the threshold.

Since a self-join produces both (A,B) and (B,A) for every pair, we add a filter to only keep rows where page_id_a < page_id_b. The result has five columns: page_id_a, title_a, page_id_b, title_b, jaccardDist. We write it to Parquet with Snappy compression. Parquet was a natural choice because it is columnar and Spark reads it very efficiently.

6.5 Evaluation Framework

evaluate.py has two separate modes that we use for different kinds of analysis.

Scalability Study:

In this mode the pipeline runs at several sample fractions (we used 1%, 2%, 5%, and 10%). For each run it records how many articles were in the sample, how many candidate pairs were found, and the time spent on each major stage: ingestion plus sampling, featurisation, and LSH fitting plus self-join. Everything gets dumped to a JSON file that we later used to make the plots in this report.

Accuracy Evaluation:

Here we take a small sample (400 documents by default, because the brute-force part is O(n^2) and takes forever on anything larger) and compute exact Jaccard distances between every pair using the binary feature vectors. Any pair at or below the distance threshold becomes part of the ground truth. Then we run LSH on the same sample and compare the two sets of pairs. The output is precision, recall, F1, and counts of true positives, false positives, and false negatives.

jaccard_utils.py has the helper functions for this: Jaccard similarity and distance on sets of feature indices, a brute-force all-pairs function, and precision/recall/F1 computation.

7. Experimental Setup

Component	Specification
Programming Language	Python 3.10 - 3.12
Distributed Framework	Apache Spark 3.4 - 3.5 (PySpark)
ML Library	Spark MLlib (MinHashLSH)

XML Parsing	spark-xml (com.databricks:spark-xml_2.12:0.18.0)
Java Runtime	OpenJDK 11 / 17
Output Format	Apache Parquet with Snappy compression
Testing Framework	pytest
Version Control	Git + GitHub

The default hyperparameters for our experiments are listed below:

Parameter	Default Value	Description
num_features	2^18 (262,144)	Dimensionality of the HashingTF vector space
min_token_length	2	Minimum token length after tokenisation
num_hash_tables	5	Number of LSH hash tables (bands)
jaccard_distance_threshold	0.3	Maximum Jaccard distance for candidate pairs
sample_seed	42	Random seed for reproducible sampling
max_docs_bruteforce	400	Maximum documents for exact brute-force evaluation

8. Results and Analysis

8.1 Scalability Analysis

We ran the pipeline at four sample sizes: 1%, 2%, 5%, and 10% of the articles in the shard. For each run we timed three stages separately: (i) ingestion and sampling, (ii) tokenisation

plus feature extraction, and (iii) LSH model fitting and the self-join that produces candidate pairs.

Ingestion and featurisation both scaled roughly linearly, which was expected since they process each document independently. The LSH self-join was a different story. Its runtime grew faster than linear because as you add more documents the number of potential candidate pairs increases. But even so, the growth was nowhere near quadratic, which is the whole point of using LSH in the first place.

On a multi-core laptop, Spark processed a few thousand articles per minute. The slowest part was actually the bzip2 decompression, which is single-threaded for a single file. In a proper HDFS setup the data would already be split across blocks and this bottleneck would disappear.

8.2 Accuracy Evaluation

For the accuracy experiments we capped the sample at 400 documents to keep the brute-force computation feasible (400 documents means about 80,000 pairs, which is manageable). We computed exact Jaccard distances for all those pairs using the binary feature vectors, and any pair with distance <= 0.3 went into the ground truth set.

We then ran LSH with the same parameters on the same sample. The LSH output matched the ground truth well. Precision and recall were both high with 5 hash tables and a 0.3 threshold. The few misses were all borderline pairs sitting near the threshold boundary, which is where the LSH S-curve transitions from high to low detection probability.

This is consistent with what the theory predicts. MinHash LSH is very good at catching pairs that are solidly within the similarity region, and the errors tend to concentrate at the boundary where the similarity is close to the threshold value.

9. Testing Strategy

We set up two levels of tests, mainly because spinning up Spark for every test run takes a while and we did not want that overhead during regular development.

Unit Tests (test_evaluate_pure.py):

These are pure Python tests that do not need Spark at all. They check four things: Jaccard similarity of two identical sets returns 1.0, Jaccard of completely different sets returns 0.0, the

brute-force function correctly picks out close pairs below a given threshold, and the precision/recall/F1 function gives correct output for hand-calculated TP/FP/FN values. They run in under a second.

Integration Tests (test_spark_smoke.py):

This test boots up a local Spark session and runs the full pipeline on a tiny XML file we created by hand. The file has three articles: two that are similar to each other and one redirect. The test checks that the parser reads all three, that the redirect gets filtered out, and that LSH finds at least one candidate pair between the two similar articles. We gate it behind the RUN_SPARK_INTEGRATION environment variable so it only runs when you explicitly ask for it.

10. How to Reproduce

Prerequisites:

- Python 3.10, 3.11, or 3.12 (Python 3.13 is not recommended due to distutils compatibility issues with PySpark 3.5)
- Java 11 or Java 17
- Internet access on first run (Spark downloads the spark-xml JAR from Maven Central)

Setup:

```
git clone
https://github.com/Yoursuperherobasu/BigData_Project_Machine_Learning-
_for_Big_Data.git
cd BigData_Project_Machine_Learning-_for_Big_Data
python3.12 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

Running the pipeline:

```
chmod +x scripts/spark_submit.sh
export SPARK_MASTER=local[*]

./scripts/spark_submit.sh \
  --input file:///absolute/path/to/enwiki-dump.xml.bz2 \
  --output file:///absolute/path/to/output_pairs \
```

```
  --sample-fraction 0.05 \
  --num-partitions 64
```

Running tests:

```
# Unit tests (fast, no Spark needed)
pytest tests/test_evaluate_pure.py -q

# Integration tests (requires local Spark)
export RUN_SPARK_INTEGRATION=1
pytest tests/test_spark_smoke.py -v
```

11. Challenges and Lessons Learned

Building this pipeline was not entirely smooth. We ran into several issues that took some time to debug, and we think they are worth documenting here.

XML Parsing at Scale: The MediaWiki XML schema is more complex than it looks. Article text is nested inside revision elements, and depending on which dump file you use, those revision elements can be a Struct or an Array in Spark's inferred schema. We spent a good amount of time on this until we added runtime schema inspection to the ingest module. We also initially tried Python's ElementTree library for parsing but abandoned that quickly because it does not integrate with Spark's distributed model.

Python and PySpark Version Compatibility: This was probably the most frustrating issue. One of us had Python 3.13 installed, and Spark kept throwing cryptic JVM classpath errors on startup. It took a while to trace this to distutils, which Python 3.13 removes but PySpark 3.5 still needs. Switching to Python 3.12 with setuptools installed fixed it. We now have a note about this at the top of the README.

Memory Management: Our first runs on larger samples kept crashing with out-of-memory errors. The issue was that Spark was recomputing intermediate DataFrames because we had not cached them. Once we started caching after expensive steps and calling unpersist right after the next stage consumed the data, the pipeline ran stably. It sounds obvious in hindsight but it was not at all obvious when we were staring at Spark error logs at 2 AM.

Git LFS and Large Files: The Wikipedia shard is 380 MB, way over GitHub's 100 MB limit. We thought about using Git LFS but decided it was simpler to just exclude the data file and document the download URL in the README. Most research projects do it this way anyway.

12. Conclusion and Future Work

In this project we put together a distributed pipeline that detects near-duplicate articles in a Wikipedia dump. It uses Spark for the distributed computation and MinHash LSH for fast candidate generation. The combination makes it feasible to handle data volumes that are completely out of reach for brute-force methods.

Our scaling experiments showed that the per-document stages grow more or less linearly with the data, and while the LSH self-join grows faster, it stays well within practical limits. The accuracy experiments on a controlled sample showed that LSH with our default parameters gives high precision and recall relative to the brute-force ground truth. The codebase is modular, has tests at two levels, and comes with enough documentation to be reproducible.

There are several things we would like to try if we had more time:

- Process the full English Wikipedia dump on an actual HDFS cluster and see whether the scaling trends hold up at millions of articles.
- Try character-level n-gram shingles instead of word tokens. They might do a better job at catching duplicates that differ only by minor edits or rephrasing.
- Do a systematic grid search over the number of hash tables and bands, plotting precision-recall curves to find the best parameter combinations.
- Hook up Spark Structured Streaming so that new articles can be checked for duplicates incrementally rather than reprocessing the whole corpus each time.
- Benchmark MinHash LSH against SimHash and against dense-embedding-based approximate nearest neighbour methods on the same dataset.

13. References

[1] Broder, A. Z. (1997). On the resemblance and containment of documents. In Proceedings of the Compression and Complexity of Sequences, pp. 21-29. IEEE.

[2] Broder, A. Z., Charikar, M., Frieze, A. M., & Mitzenmacher, M. (1998). Min-wise independent permutations. In Proceedings of the 30th Annual ACM Symposium on Theory of Computing (STOC), pp. 327-336.

[3] Indyk, P., & Motwani, R. (1998). Approximate nearest neighbors: towards removing the curse of dimensionality. In Proceedings of the 30th Annual ACM Symposium on Theory of Computing (STOC), pp. 604-613.

[4] Henzinger, M. (2006). Finding near-duplicate web pages: a large-scale evaluation of algorithms. In Proceedings of the 29th Annual International ACM SIGIR Conference, pp. 284-291.

[5] Manku, G. S., Jain, A., & Sarma, A. D. (2007). Detecting near-duplicates for web crawling. In Proceedings of the 16th International Conference on World Wide Web (WWW), pp. 141-150.

[6] Leskovec, J., Rajaraman, A., & Ullman, J. D. (2020). Mining of Massive Datasets. Cambridge University Press. Chapter 3: Finding Similar Items.

[7] Apache Spark Documentation. MLlib - MinHashLSH. https://spark.apache.org/docs/latest/ml-features.html#minhash-for-jaccard-distance

[8] Zaharia, M., Chowdhury, M., Franklin, M. J., Shenker, S., & Stoica, I. (2010). Spark: Cluster Computing with Working Sets. In Proceedings of the 2nd USENIX Conference on Hot Topics in Cloud Computing.

[9] Wikimedia Foundation. Wikimedia Downloads: Database Dumps. https://dumps.wikimedia.org